Lesson 2: functions and object-oriented programming

The course uses C++17 as its maximum language standard. Teaching code uses explicit types, equals initialization for scalar values, parentheses for ordinary object construction, and initializer lists after equals for arrays and aggregate data.

These are continuous lecture notes following the original repository's demonstration sequence. Each experiment includes conceptual reasoning, a complete single-file program, a state trace, failure analysis, and a self-check with an answer. A separate bank contains 120 exercises; function and pointer practice follows the original Lesson 2 placement.

Use CPP-Classroom with C++17 and paste one complete .cpp program. The platform executes code but does not perform this course's automatic grading. Prefill stdin or send lines interactively; send EOF for programs that read until end-of-input.

Original repository: unnc-aim/aim-2526-rookie-courses

Contents

- 01. Functions: arguments, computation, and return
- 02. References, addresses, and pointers
- 03. Understanding parameter passing through swapping
- 04. From struct to the Band class
- 05. Constructing and initializing Band
- 06. Instances, methods, and a collection of bands
- 07. MofoBand: inheritance and base construction
- 08. protected, this, and a second derived layer
- 09. Original-course integration and engineering extensions

01 Functions: arguments, computation, and return

Original source: test_add, the original Lesson 2 opening

1.1 Reading the test_add interface

Lesson 2 keeps the original test_add opening rather than starting with inheritance or resource management. Its declaration states the complete interface: two int parameters and an int result. Calling test_add(1,2) initializes the local parameters a and b from the arguments, then executes the function body in its own scope.

The original prints a demonstration message, computes sum, and returns it. Output and return values are separate channels: cout writes to a stream, whereas return supplies the result to the caller. Printing 3 does not mean the call expression returns 3.

The caller may store the returned value or use it in another expression. The local sum ceases to exist after the function returns, but its integer value has already been delivered to the caller. Returning a number does not require a global variable.

Complete classroom experiment

`L2_C01_D01.cpp` · CPP-Classroom source

```
#include <iostream>

int test_add(int a, int b) {
    int sum = a + b;
    return sum;
}

int main() {
    int result = test_add(1, 2);
    std::cout << result << '\n';
    return 0;
}
```

Execution trace

Inside the function, a=1, b=2, and sum=3. Returning delivers 3 to the call expression, initializes result, and the caller prints 3.

Common error and its cause

An ordinary int-returning function needs an appropriate result on paths that return. Do not generalize main’s special end-of-function rule to other functions.

Self-check

What value does std::cout << test_add(2,3) obtain before printing?

Answer: The call produces the integer 5, which the outer stream prints. Any output inside the function also occurs.

1.2 Declarations, definitions, and void

The compiler needs a declaration at a call site. Defining the function before main is simplest in introductory examples. A later definition requires an earlier declaration. A declaration ends with a semicolon; a definition provides the body. Their names and types must match.

void means no result value is returned; it does not mean the function does nothing. The original cite_pointer, swap1, swap2, and print_info produce output or mutate state. A function may have no parameters, or parameters without a returned value. Choose the interface for its responsibility.

Classroom programs keep declarations, definitions, and main in one source file. The original header/source separation is explained as project organization, without requiring a multi-file build that the current platform does not provide.

Complete classroom experiment

`L2_C01_D02.cpp` · CPP-Classroom source

```
#include <iostream>

int test_add(int a, int b) { return a + b; }
void print_sum(int a, int b) { std::cout << test_add(a, b) << '\n'; }

int main() {
    print_sum(4, 5);
    return 0;
}
```

Execution trace

print_sum takes two integers, calls test_add, and prints the result. The caller receives no value because print_sum returns void.

Common error and its cause

A declaration without a matching implementation can lead to a link error, unlike a call with incompatible types that fails compilation.

Self-check

Can a void function use return; to finish early?

Answer: Yes. A return without a value ends that invocation.

Chapter exercises

ID	Exercise
S49	Parameters and a return value
S50	Declaration and definition
S51	A clamping function
S52	Default arguments
S53	Function overloading
S54	Local scope
S55	Recursive factorial
S56	Boolean success and an output parameter

02 References, addresses, and pointers

Original source: cite_pointer: band1, live1, anime1

2.1 A reference aliases an existing object

The original cite_pointer defines band1 as "poppin party" and binds string& live1 to it. live1 is not a copied string; it names the same object. A write through live1 changes what is subsequently read through band1.

A reference is normally bound when declared. Assigning live1=another assigns another's value to the bound object; it does not rebind live1 to a different object. This distinction matters for swapping and passing objects to functions.

The original prints &band1 and &live1 to show that the names refer to the same object. Actual addresses vary across runs. We use the stable boolean comparison &band1==&live1 for sample verification, while allowing instructors to inspect the addresses separately.

Complete classroom experiment

`L2_C02_D01.cpp` · CPP-Classroom source

```
#include <iostream>
#include <string>

int main() {
    std::string band1 = "poppin party";
    std::string &live1 = band1;
    live1 = "afterglow";
    std::cout << band1 << '\n';
    std::cout << std::boolalpha << (&band1 == &live1) << '\n';
    return 0;
}
```

Execution trace

The output is afterglow and true. A single string object is updated; there is no second live1 string to synchronize.

Common error and its cause

Changing std::string& to std::string creates a copy, so later assignment no longer modifies band1.

Self-check

If the & is removed from the declaration, does band1 become afterglow?

Answer: No. live1 becomes independent, and band1 remains poppin party.

2.2 Pointer variables and dereferencing

The original initializes `string*` anime1 to nullptr, then assigns &band1 to it. A pointer is an object that stores an address value. After receiving the address of band1, anime1 refers to that string. Its type `string*` differs from the string object's type.

In an expression, `*anime1` dereferences the pointer and accesses its target. Streaming anime1 displays an address representation; streaming `*anime1` displays the string. The star in a declaration and the star in a dereference have different grammatical roles.

Assigning through `*anime1` changes the target string without changing the stored address. Assigning anime1=&band2 redirects the pointer without changing band1. Track the pointer and its target as separate state variables.

Complete classroom experiment

`L2_C02_D02.cpp` · CPP-Classroom source

```
#include <iostream>
#include <string>

int main() {
    std::string band1 = "poppin party";
    std::string band2 = "mygo";
    std::string *anime1 = &band1;
    *anime1 = "afterglow";
    anime1 = &band2;
    std::cout << band1 << ' ' << *anime1 << '\n';
    return 0;
}
```

Execution trace

The dereference changes band1 to afterglow, then the pointer is redirected to band2. The output is afterglow mygo; redirecting the pointer does not undo the earlier write.

Common error and its cause

An uninitialized pointer is not guaranteed to be nullptr and cannot be dereferenced safely.

Self-check

What does p=&x modify, and what does *p=x modify?

Answer: The former changes the address stored in p; the latter changes p's current target.

2.3 Null pointers, read-only borrowing, and lifetime

nullptr denotes no target object and cannot be dereferenced. The original dereferences only after assigning a valid address; we make that precondition explicit. A nullable interface needs a check. A reference can express a required target, but that target must still remain alive.

const string& borrows a string for reading without copying it. const `string*` prevents modification through that pointer but can be redirected; `string* const` fixes the pointer itself while its target may remain mutable. Identify which layer is const before reasoning about mutability.

An ordinary local object is destroyed when its function scope ends. Returning its address or a reference leaves a dangling access path. Return computed values or new strings by value instead of avoiding a copy at the cost of an invalid lifetime.

Complete classroom experiment

`L2_C02_D03.cpp` · CPP-Classroom source

```
#include <iostream>
#include <string>

int main() {
    std::string band = "mygo";
    const std::string *p = nullptr;
    if (p != nullptr) {
        std::cout << *p << '\n';
    }
    p = &band;
    std::cout << *p << '\n';
    return 0;
}
```

Execution trace

The first check skips dereferencing a null pointer. After p receives band’s address, it prints mygo. The pointer borrows band; no dynamic allocation occurs and the address must not be deleted.

Common error and its cause

A non-null address does not prove that the target is still alive. Dangling pointers may be non-null.

Self-check

Does returning a const reference to a local string extend its lifetime?

Answer: No. const does not preserve a local object destroyed when the function exits.

Chapter exercises

ID	Exercise
S59	Read and write through a pointer
S60	Guard a null pointer
S61	Pointer to const
S62	Read-only reference traversal
S63	Return by value to avoid dangling
S64	A non-owning pointer range

03 Understanding parameter passing through swapping

Original source: swap1; swap2; reference swap as a comparison

3.1 Why swap1 does not exchange the caller's values

The original swap1(int a,int b) uses a temporary and three assignments. It really does exchange a and b, but those parameters are integer copies of the arguments. The caller's x and y are different objects and are unchanged.

Trace four values: outside x=10 and y=20; inside a=10 and b=20. The temporary saves 10, a becomes 20, and b becomes 10. When the call ends, those locals disappear, leaving x and y unchanged. Printing only inside the function can hide this distinction.

This is an interface choice, not a compiler defect or a faulty temporary. Value parameters suit calculations on input copies. To mutate caller-owned objects, the interface must borrow those objects through references or pointers.

Complete classroom experiment

`L2_C03_D01.cpp` · CPP-Classroom source

```
#include <iostream>

void swap1(int a, int b) {
    int temp = a;
    a = b;
    b = temp;
}

int main() {
    int x = 10, y = 20;
    swap1(x, y);
    std::cout << x << ' ' << y << '\n';
    return 0;
}
```

Execution trace

The output is 10 20 regardless of the final local values a=20 and b=10.

Step	State / operation	Result
`call`	`x=10,y=20`	`a=10,b=20`
`temp=a`	`x=10,y=20`	`temp=10`
`a=b`	`x=10,y=20`	`a=20`
`b=temp`	`x=10,y=20`	`b=10`
`return`	`x=10,y=20`	`parameters destroyed`

Common error and its cause

A changed parameter value does not prove that the caller's object changed. Observe both scopes.

Self-check

If swap1 returns int but the caller does not assign the result to x or y, will swapping happen automatically?

Answer: No. A return type does not change value-parameter semantics.

3.2 How swap2 modifies original objects through addresses

The original swap2 receives `int*` parameters and is called with &x and &y. The pointer parameters are still passed by value, but their copied address values designate the original objects. Writing `*p1` modifies x. It is inaccurate to say pointer parameters are not passed by value.

The three assignments now operate on targets: save the original x, copy the original y into x, and then write the temporary into y. Both caller objects change. The pointers must designate valid writable ints; a nullable interface needs explicit null behavior.

A reference version swaps original objects with direct variable syntax and does not require & at the call site. Choose based on nullability, pointer redirection, and interface conventions. In either form, be able to identify exactly which object is modified.

Complete classroom experiment

`L2_C03_D02.cpp` · CPP-Classroom source

```
#include <iostream>

void swap2(int *p1, int *p2) {
    int temp = *p1;
    *p1 = *p2;
    *p2 = temp;
}
void swap_ref(int &a, int &b) {
    int temp = a;
    a = b;
    b = temp;
}

int main() {
    int x = 10, y = 20;
    swap2(&x, &y);
    std::cout << x << ' ' << y << '\n';
    swap_ref(x, y);
    std::cout << x << ' ' << y << '\n';
    return 0;
}
```

Execution trace

The pointer swap produces 20 10; the reference swap restores 10 20. Both calls modify the same caller-owned objects.

Step	State / operation	Result
`call`	`p1=&x,p2=&y`	`x=10,y=20`
`temp=*p1`	`temp=10`	`x=10,y=20`
`*p1=*p2`	`write x`	`x=20,y=20`
`*p2=temp`	`write y`	`x=20,y=10`

Common error and its cause

Swapping the local pointers p1 and p2 does not exchange their target values x and y.

Self-check

Does swap2(&x,&x) corrupt x?

Answer: Not with this three-assignment algorithm. Both pointers target the same valid object, and its value is preserved.

Chapter exercises

ID	Exercise
S57	Value parameters do not swap callers
S58	Swap through references

04 From struct to the Band class

Original source: Band, retaining the original band-member scenario

4.1 Why a data record may need a behavioral boundary

The original Band groups guitar, vocal, drum, bass, and keyboard in one type. This extends the Lesson 1 structure: each Band object represents one band, and each field stores part of its state. A single type can create many bands instead of relying on one global lineup.

Public fields allow arbitrary external writes. private restricts direct access, while public member functions define permitted reads and changes. Encapsulation is about enforcing state rules, not mechanically producing getters and setters for every field.

Both struct and class support member functions, constructors, and inheritance. Their principal language differences are default member access and default inheritance access: public for struct and private for class. The original transition is pedagogical, not evidence that struct lacks object-oriented features.

Complete classroom experiment

`L2_C04_D01.cpp` · CPP-Classroom source

```
#include <iostream>
#include <string>

struct BandRecord {
    std::string guitar, vocal, bass;
};

int main() {
    BandRecord band = {"mutsumi", "hatsune", "umiri"};
    std::cout << band.guitar << ':' << band.vocal << '\n';
    return 0;
}
```

Execution trace

This transitional record still allows direct field access. The next section places state behind private access and changes how callers use it.

Common error and its cause

Using the class keyword does not automatically provide useful encapsulation if callers can still modify every field.

Self-check

Why does external field access fail after changing struct to class without adding public?

Answer: The default member access changes from public to private.

4.2 private, public, and calling member functions

Read the access labels before examining individual members. The five strings under private are internal state; constructors and methods under public form the external interface. band.guitar is rejected outside the class, but band.get_guitar() calls a method that may read that state.

The object before the dot determines which band is affected. Calling a.set_keyboard_anon() changes a, not b. Inside a member function, guitar refers to the current object's field without an explicit object prefix; this later makes that connection explicit.

The original comment says a constructor must be public. That is not a universal rule. This example uses a public constructor so main can create Band directly; other designs may restrict creation with a private constructor and a factory.

Complete classroom experiment

`L2_C04_D02.cpp` · CPP-Classroom source

```
#include <iostream>
#include <string>
#include <utility>

class Band {
  private:
    std::string guitar, vocal, drum, bass, keyboard;

  public:
    Band(std::string gt, std::string vo, std::string dr, std::string ba,
         std::string key) {
        guitar = std::move(gt);
        vocal = std::move(vo);
        drum = std::move(dr);
        bass = std::move(ba);
        keyboard = std::move(key);
    }
    void print_info() const {
        std::cout << guitar << ',' << vocal << ',' << drum << ',' << bass << ','
                  << keyboard << '\n';
    }
    void live() const { std::cout << "LIVE\n"; }
    void get_guitar() const { std::cout << guitar << '\n'; }
    void set_keyboard_anon() { keyboard = "anon"; }
};

int main() {
    Band a("mutsumi", "hatsune", "nyamu", "umiri", "saki");
    Band b("hina", "aya", "maya", "chisato", "eve");
    a.set_keyboard_anon();
    a.print_info();
    b.print_info();
    return 0;
}
```

Execution trace

a's final field becomes anon, while b's remains eve. Comparing two instances distinguishes a shared type definition from independent object state.

Step	State / operation	Result
`construct a`	`a.keyboard`	`saki`
`construct b`	`b.keyboard`	`eve`
`a.set_keyboard_anon()`	`a.keyboard`	`anon`
`observe b`	`b.keyboard`	`eve`

Common error and its cause

Keep deliberate private-access errors in a separate experiment rather than in the runnable reference program.

Self-check

Does get_guitar reading a private field mean that private is ineffective?

Answer: No. private restricts external direct access; the class's methods are the intended place to manage its fields.

Chapter exercises

ID	Exercise
O01	A first class
O02	Independent object state
O03	A private speed boundary
O04	Reject an invalid mutation
O05	struct and class defaults

05 Constructing and initializing Band

Original source: Band construction; std::move; preparation for base initialization

5.1 How constructor arguments become object state

Band construction takes five strings corresponding to its five members. A constructor has the class name and no return type. It runs as the object is created and establishes usable initial state. Ordinary methods act on an object that already exists.

The original default-constructs five strings and then assigns them in the constructor body. This works for string but is not a general solution. const members, references, and members lacking default constructors need appropriate initialization before the body begins.

The original uses std::move on parameters to permit transferring their resources to members. std::move is a cast; it does not itself move bytes. The later operation and type determine whether a move occurs. Here the local parameter's original contents are no longer needed after assignment.

Complete classroom experiment

`L2_C05_D01.cpp` · CPP-Classroom source

```
#include <iostream>
#include <string>
#include <utility>

class Band {
  private:
    std::string guitar, vocal, drum, bass, keyboard;

  public:
    Band(std::string gt, std::string vo, std::string dr, std::string ba,
         std::string key) {
        guitar = std::move(gt);
        vocal = std::move(vo);
        drum = std::move(dr);
        bass = std::move(ba);
        keyboard = std::move(key);
    }
    void print_info() const {
        std::cout << guitar << ',' << vocal << ',' << drum << ',' << bass << ','
                  << keyboard << '\n';
    }
    void live() const { std::cout << "LIVE\n"; }
    void get_guitar() const { std::cout << guitar << '\n'; }
    void set_keyboard_anon() { keyboard = "anon"; }
};

int main() {
    Band band("mutsumi", "hatsune", "nyamu", "umiri", "saki");
    band.print_info();
    return 0;
}
```

Execution trace

The arguments initialize parameters, which populate the corresponding members. print_info determines the output order explicitly.

Common error and its cause

Do not place void before a constructor name. Constructors do not have a return type.

Self-check

What happens if Band is constructed with only four arguments?

Answer: Compilation fails because this interface has no matching four-argument constructor.

5.2 Member initializer lists and actual initialization order

A member initializer list after the constructor's colon directly initializes members before the body starts. In Band(...):guitar(move(gt)), the parentheses provide the member initializer; guitar is not being called as an ordinary function.

Members initialize in their declaration order, not the order written in the initializer list. That matters when one initializer depends on another member. Base subobjects are initialized before derived-class members, preparing the later MofoBand(...):Band(...) example.

First understand what must be ready as an object is established. Delegating constructors, move constructors, and other special members are extensions, not replacements for the original path from basic construction to inheritance.

Complete classroom experiment

`L2_C05_D02.cpp` · CPP-Classroom source

```
#include <iostream>
#include <string>
#include <utility>

class BandInit {
    std::string guitar;
    std::string keyboard;

  public:
    BandInit(std::string gt, std::string key)
        : guitar(std::move(gt)), keyboard(std::move(key)) {}
    void print() const { std::cout << guitar << ':' << keyboard << '\n'; }
};

int main() {
    BandInit band("mutsumi", "saki");
    band.print();
    return 0;
}
```

Execution trace

guitar and keyboard are initialized before the empty body starts, producing mutsumi:saki.

Common error and its cause

An initializer list cannot appear inside the constructor body. The colon belongs after the parameter list and before the body.

Self-check

Does listing keyboard before guitar change their actual initialization order?

Answer: No. Declaration order still controls initialization; matching that order avoids misleading readers.

Chapter exercises

ID	Exercise
O06	Member initializer list
O07	A const data member
O08	Delegating construction
O09	Prevent implicit conversion
O10	A non-default-constructible member

06 Instances, methods, and a collection of bands

Original source: class_band in its original execution order

6.1 Read methods, write methods, and const

The original class_band creates ave_mujica, calls print_info and live, and later uses get_guitar and set_keyboard_anon. Some methods observe state; others mutate it. Their call order determines what later observations return.

We mark observational methods const. The const after a member-function parameter list prevents modification of ordinary members through that invocation and allows calls on const Band. It does not qualify the return value or freeze all instances of the class.

The original get_guitar prints a name rather than returning a string. Preserve that distinction when using the original interface. A production interface could return the name for further processing, but a method's name does not determine its return behavior.

Complete classroom experiment

`L2_C06_D01.cpp` · CPP-Classroom source

```
#include <iostream>
#include <string>
#include <utility>

class Band {
  private:
    std::string guitar, vocal, drum, bass, keyboard;

  public:
    Band(std::string gt, std::string vo, std::string dr, std::string ba,
         std::string key) {
        guitar = std::move(gt);
        vocal = std::move(vo);
        drum = std::move(dr);
        bass = std::move(ba);
        keyboard = std::move(key);
    }
    void print_info() const {
        std::cout << guitar << ',' << vocal << ',' << drum << ',' << bass << ','
                  << keyboard << '\n';
    }
    void live() const { std::cout << "LIVE\n"; }
    void get_guitar() const { std::cout << guitar << '\n'; }
    void set_keyboard_anon() { keyboard = "anon"; }
};

int main() {
    Band ave_mujica("mutsumi", "hatsune", "nyamu", "umiri", "saki");
    ave_mujica.print_info();
    ave_mujica.live();
    ave_mujica.get_guitar();
    ave_mujica.set_keyboard_anon();
    ave_mujica.print_info();
    return 0;
```

```
}
```

Execution trace

The initial lineup ends with saki. The next outputs are LIVE and mutsumi. Only after the setter does the final lineup end with anon.

Common error and its cause

You cannot assign the result of void get_guitar() to a string because it returns no value.

Self-check

If ave_mujica is const, which call is rejected?

Answer: set_keyboard_anon is rejected because it mutates state; the const observational methods remain callable.

6.2 `vector<Band>`, emplace_back, and reference iteration

The original live_bands is a `vector<Band>`: its elements are objects of our own type rather than integers. emplace_back forwards the five names needed to construct a Band in the collection. This stores objects with behavior, not one concatenated text lineup.

The original for(Band& p:live_bands) references each stored object rather than copying a band every iteration. For printing, const Band& expresses read-only access and also works with a const collection. Use a mutable reference when the loop intentionally changes objects.

Using Band p copies an element each iteration. A setter applied to that copy does not change the stored band. This is the same value-copy behavior seen in array iteration and swap1's value parameters.

Complete classroom experiment

`L2_C06_D02.cpp` · CPP-Classroom source

```
#include <iostream>
#include <string>
#include <utility>
#include <vector>

class Band {
  private:
    std::string guitar, vocal, drum, bass, keyboard;

  public:
    Band(std::string gt, std::string vo, std::string dr, std::string ba,
         std::string key) {
        guitar = std::move(gt);
        vocal = std::move(vo);
        drum = std::move(dr);
        bass = std::move(ba);
        keyboard = std::move(key);
```

```
    }
    void print_info() const {
        std::cout << guitar << ',' << vocal << ',' << drum << ',' << bass << ','
                  << keyboard << '\n';
    }
    void live() const { std::cout << "LIVE\n"; }
    void get_guitar() const { std::cout << guitar << '\n'; }
    void set_keyboard_anon() { keyboard = "anon"; }
};

int main() {
    std::vector<Band> live_bands;
    live_bands.emplace_back("hina", "aya", "maya", "chisato", "eve");
    live_bands.emplace_back("sayo", "yukina", "ako", "lisa", "rinko");
    for (const Band &p : live_bands) {
        p.print_info();
    }
    return 0;
}
```

Execution trace

The vector holds two bands and prints them in insertion order. Each iteration refers to the current element rather than creating an independently mutable copy.

Common error and its cause

Growth may relocate elements. Do not retain old element addresses across growth or append casually while reference-iterating.

Self-check

Does replacing const Band& with Band necessarily change the printed result?

Answer: Read-only output may look identical, but copying occurs. Mutation exposes the state difference.

Chapter exercises

ID	Exercise
O11	A const member function
O12	this and a shadowed parameter
O13	Return a reference for chaining
O14	Copies and read-only views
O15	A static member function
O16	Copy an object by value
O17	Object parameters by value and reference
O18	Scope and destruction order
O19	Store objects in a vector
O20	Count live objects

07 MofoBand: inheritance and base construction

Original source: MofoBand : public Band; print_mofo_info

7.1 What public inheritance establishes

The original MofoBand adds vocal_actor to the five-member Band information. Public inheritance expresses a usable Band relationship and makes inherited public methods available. A derived object contains a Band base subobject; inheritance is not textual copying of source code.

Band's private fields remain under Band's control. MofoBand can call print_info but cannot directly write guitar. Inheritance does not bypass encapsulation or grant arbitrary private access; needed behavior goes through the base interface.

Check the relationship before choosing inheritance. The band extension serves this model, while a robot usually owns a motor through composition rather than inheriting from Motor. The later robot exercises are labeled as applications of these distinct relationships.

Complete classroom experiment

`L2_C07_D01.cpp` · CPP-Classroom source

```
#include <iostream>
#include <string>
#include <utility>

class Band {
  private:
    std::string guitar, vocal, drum, bass, keyboard;

  public:
    Band(std::string gt, std::string vo, std::string dr, std::string ba,
         std::string key) {
        guitar = std::move(gt);
        vocal = std::move(vo);
        drum = std::move(dr);
        bass = std::move(ba);
        keyboard = std::move(key);
    }
    void print_info() const {
        std::cout << guitar << ',' << vocal << ',' << drum << ',' << bass << ','
                  << keyboard << '\n';
    }
    void live() const { std::cout << "LIVE\n"; }
    void get_guitar() const { std::cout << guitar << '\n'; }
    void set_keyboard_anon() { keyboard = "anon"; }
};
class MofoBand : public Band {
  protected:
    std::string vocal_actor;

  public:
    MofoBand(std::string gt, std::string vo, std::string dr, std::string ba,
             std::string key, std::string actor)
```

```
        : Band(std::move(gt), std::move(vo), std::move(dr), std::move(ba),
               std::move(key)),
          vocal_actor(std::move(actor)) {}
    void print_mofo_info() const {
        print_info();
        std::cout << vocal_actor << '\n';
    }
};

int main() {
    MofoBand morfonica("toko", "mashiro", "tsukushi", "nanami", "rui", "amane");
    morfonica.print_info();
    morfonica.print_mofo_info();
    return 0;
}
```

Execution trace

The first call prints the five base fields. The second prints those fields and then amane. The base state is not duplicated.

Common error and its cause

Calling an inherited public method is different from directly accessing a private base field.

Self-check

Can a MofoBand member function directly assign guitar="new"?

Answer: No. guitar is private to Band and must be managed through Band's permitted interface.

7.2 Why the initializer list calls Band

MofoBand takes six strings: five initialize the base, and the last initializes its added member. Band(...) after the colon is a base initializer, not an ordinary call statement. The base is constructed before derived members and the derived constructor body.

Band has a five-argument constructor and no usable parameterless one. This MofoBand cannot postpone base initialization into its body. The original claim that every subclass must define its own constructor is too broad: this design must supply arguments for its non-default-constructible base.

We also initialize vocal_actor in the member initializer list. Actual order is the Band base, then vocal_actor, then the derived constructor body. The position of method declarations in the class does not change this order.

Complete classroom experiment

`L2_C07_D02.cpp` · CPP-Classroom source

```
#include <iostream>
#include <string>
#include <utility>
```

```
class Band {
  private:
    std::string guitar, vocal, drum, bass, keyboard;

  public:
    Band(std::string gt, std::string vo, std::string dr, std::string ba,
         std::string key) {
        guitar = std::move(gt);
        vocal = std::move(vo);
        drum = std::move(dr);
        bass = std::move(ba);
        keyboard = std::move(key);
    }
    void print_info() const {
        std::cout << guitar << ',' << vocal << ',' << drum << ',' << bass << ','
                  << keyboard << '\n';
    }
    void live() const { std::cout << "LIVE\n"; }
    void get_guitar() const { std::cout << guitar << '\n'; }
    void set_keyboard_anon() { keyboard = "anon"; }
};
class MofoBand : public Band {
  protected:
    std::string vocal_actor;

  public:
    MofoBand(std::string gt, std::string vo, std::string dr, std::string ba,
             std::string key, std::string actor)
        : Band(std::move(gt), std::move(vo), std::move(dr), std::move(ba),
               std::move(key)),
          vocal_actor(std::move(actor)) {}
    void print_mofo_info() const {
        print_info();
        std::cout << vocal_actor << '\n';
    }
};

int main() {
    MofoBand band("toko", "mashiro", "tsukushi", "nanami", "rui", "amane");
    band.print_mofo_info();
    return 0;
}
```

Execution trace

The five-field Band and actor member are established before a method observes the complete derived object.

Step	State / operation	Result
1	`Band base`	`five member strings`
2	`MofoBand member`	`vocal_actor`
3	`constructor body`	`complete derived object`

Common error and its cause

Writing Band(...) inside the body would create a temporary, not initialize the current object's base subobject.

Self-check

What does the compiler try to find if :Band(...) is removed?

Answer: It needs to default-initialize Band, but no matching default constructor exists here.

Chapter exercises

ID	Exercise
O26	Public inheritance
O27	Initialize the base
O29	Base and derived lifetimes
O30	Name hiding and qualification

08 protected, this, and a second derived layer

Original source: MofoBand_New; change_actor

8.1 The exact role of protected access

The original places vocal_actor under protected, allowing MofoBand_New::change_actor to access it. Its comment saying the current class cannot access the member is incorrect. The declaring class can access it, and derived members have access under C++'s protected rules; ordinary external callers do not.

Access specifiers are compiler-enforced rules, not memory encryption. Exposing protected data couples derived classes to representation details. A narrower protected method or public interface may better preserve invariants in production; this lesson first explains the original inheritance chain.

Both inheritance steps are public. MofoBand_New extends MofoBand and still contains the Band base. Construction passes the five names and actor through the chain; adding change_actor does not remove the base constructor's requirements.

Complete classroom experiment

`L2_C08_D01.cpp` · CPP-Classroom source

```
#include <iostream>
#include <string>
#include <utility>

class Band {
  private:
    std::string guitar, vocal, drum, bass, keyboard;

  public:
    Band(std::string gt, std::string vo, std::string dr, std::string ba,
         std::string key) {
        guitar = std::move(gt);
        vocal = std::move(vo);
        drum = std::move(dr);
        bass = std::move(ba);
        keyboard = std::move(key);
    }
    void print_info() const {
        std::cout << guitar << ',' << vocal << ',' << drum << ',' << bass << ','
                  << keyboard << '\n';
    }
    void live() const { std::cout << "LIVE\n"; }
    void get_guitar() const { std::cout << guitar << '\n'; }
    void set_keyboard_anon() { keyboard = "anon"; }
};
class MofoBand : public Band {
  protected:
    std::string vocal_actor;

  public:
```

```
    MofoBand(std::string gt, std::string vo, std::string dr, std::string ba,
             std::string key, std::string actor)
        : Band(std::move(gt), std::move(vo), std::move(dr), std::move(ba),
               std::move(key)),
          vocal_actor(std::move(actor)) {}
    void print_mofo_info() const {
        print_info();
        std::cout << vocal_actor << '\n';
    }
};
class MofoBand_New : public MofoBand {
  public:
    MofoBand_New(std::string gt, std::string vo, std::string dr, std::string ba,
                 std::string key, std::string actor)
        : MofoBand(std::move(gt), std::move(vo), std::move(dr), std::move(ba),
                   std::move(key), std::move(actor)) {}
    void change_actor(const std::string &new_actor) {
        this->vocal_actor = new_actor;
    }
};

int main() {
    MofoBand_New band("toko", "mashiro", "tsukushi", "nanami", "rui", "amane");
    band.change_actor("ayane");
    band.print_mofo_info();
    return 0;
}
```

Execution trace

Construction sets the actor to amane. The derived method updates it to ayane, which the next observation displays.

Common error and its cause

main cannot directly assign band.vocal_actor; it calls the public change_actor method.

Self-check

Can MofoBand::print_mofo_info read its own protected vocal_actor?

Answer: Yes. protected does not prevent the declaring class from accessing its own member.

8.2 this identifies the object; const& borrows an argument

In change_actor, this points to the object receiving the current call. this->vocal_actor explicitly names that object's member and has the same effect as vocal_actor here. It can disambiguate a same-named parameter, although naming conventions can also reduce shadowing.

const string& new_actor borrows the caller's string read-only, avoiding an initial parameter copy. Assigning it to vocal_actor gives the member its own value; the class does not retain a reference to a short-lived argument. The member remains usable after the call returns.

Static methods have no implicit this because they are not bound to an individual object. Distinguish this, a pointer, from `*this`, the object it designates. Extensions can use that

distinction for chained method calls.

Complete classroom experiment

`L2_C08_D02.cpp` · CPP-Classroom source

```
#include <iostream>
#include <string>
#include <utility>

class Band {
  private:
    std::string guitar, vocal, drum, bass, keyboard;

  public:
    Band(std::string gt, std::string vo, std::string dr, std::string ba,
         std::string key) {
        guitar = std::move(gt);
        vocal = std::move(vo);
        drum = std::move(dr);
        bass = std::move(ba);
        keyboard = std::move(key);
    }
    void print_info() const {
        std::cout << guitar << ',' << vocal << ',' << drum << ',' << bass << ','
                  << keyboard << '\n';
    }
    void live() const { std::cout << "LIVE\n"; }
    void get_guitar() const { std::cout << guitar << '\n'; }
    void set_keyboard_anon() { keyboard = "anon"; }
};
class MofoBand : public Band {
  protected:
    std::string vocal_actor;

  public:
    MofoBand(std::string gt, std::string vo, std::string dr, std::string ba,
             std::string key, std::string actor)
        : Band(std::move(gt), std::move(vo), std::move(dr), std::move(ba),
               std::move(key)),
          vocal_actor(std::move(actor)) {}
    void print_mofo_info() const {
        print_info();
        std::cout << vocal_actor << '\n';
    }
};
class MofoBand_New : public MofoBand {
  public:
    MofoBand_New(std::string gt, std::string vo, std::string dr, std::string ba,
                 std::string key, std::string actor)
        : MofoBand(std::move(gt), std::move(vo), std::move(dr), std::move(ba),
                   std::move(key), std::move(actor)) {}
    void change_actor(const std::string &new_actor) {
        this->vocal_actor = new_actor;
    }
};

int main() {
    MofoBand_New a("toko", "mashiro", "tsukushi", "nanami", "rui", "amane");
    {
        std::string replacement = "ayane";
        a.change_actor(replacement);
```

```
    }
    a.print_mofo_info();
    return 0;
}
```

Execution trace

After replacement goes out of scope, a still stores ayane because the assignment gave the member its own string value.

Common error and its cause

A reference member bound to a short-lived argument would have different lifetime requirements.

Self-check

Does a.change_actor also change the actor of an independent object b?

Answer: No. this identifies a, the receiver of this invocation.

Chapter exercises

ID	Exercise
O28	Protected access

09 Original-course integration and engineering extensions

Original source: class_band integration, followed by explicitly labeled extensions

9.1 Reading class_band as a state trace

The original main enables class_band while other demonstrations are toggled by comments. We provide separate single-file experiments to avoid hidden toggle state. The integration keeps the same order: create and inspect a band, build and iterate a collection, read and update state, construct a derived band, and update an actor through the second derived layer.

Track four things while reading: type, variable name, current state, and callable interface. The type determines available operations, the variable identifies the instance, method calls change state, and access rules determine valid expressions. This is more useful for debugging than merely paraphrasing each line.

You should now explain the roles of the original functions and classes and predict output after changing inputs. Understanding test_add, the alias and pointer, both swap forms, and the three band classes is the priority before introducing more advanced terminology.

Complete classroom experiment

`L2_C09_D01.cpp` · CPP-Classroom source

```
#include <iostream>
#include <string>
#include <utility>

class Band {
  private:
    std::string guitar, vocal, drum, bass, keyboard;

  public:
    Band(std::string gt, std::string vo, std::string dr, std::string ba,
         std::string key) {
        guitar = std::move(gt);
        vocal = std::move(vo);
        drum = std::move(dr);
        bass = std::move(ba);
        keyboard = std::move(key);
    }
    void print_info() const {
        std::cout << guitar << ',' << vocal << ',' << drum << ',' << bass << ','
                  << keyboard << '\n';
    }
    void live() const { std::cout << "LIVE\n"; }
    void get_guitar() const { std::cout << guitar << '\n'; }
    void set_keyboard_anon() { keyboard = "anon"; }
};
class MofoBand : public Band {
  protected:
    std::string vocal_actor;

  public:
```

```
    MofoBand(std::string gt, std::string vo, std::string dr, std::string ba,
             std::string key, std::string actor)
        : Band(std::move(gt), std::move(vo), std::move(dr), std::move(ba),
               std::move(key)),
          vocal_actor(std::move(actor)) {}
    void print_mofo_info() const {
        print_info();
        std::cout << vocal_actor << '\n';
    }
};
class MofoBand_New : public MofoBand {
  public:
    MofoBand_New(std::string gt, std::string vo, std::string dr, std::string ba,
                 std::string key, std::string actor)
        : MofoBand(std::move(gt), std::move(vo), std::move(dr), std::move(ba),
                   std::move(key), std::move(actor)) {}
    void change_actor(const std::string &new_actor) {
        this->vocal_actor = new_actor;
    }
};

int main() {
    Band basic("mutsumi", "hatsune", "nyamu", "umiri", "saki");
    basic.print_info();
    basic.set_keyboard_anon();
    basic.print_info();
    MofoBand_New derived("toko", "mashiro", "tsukushi", "nanami", "rui", "amane");
    derived.change_actor("ayane");
    derived.print_mofo_info();
    return 0;
}
```

Execution trace

Observe the base object before and after its update, then the derived object's lineup and actor. Independent object state must not leak across instances.

Common error and its cause

Final output alone may not locate an incorrect intermediate update. Add checkpoints after meaningful transitions.

Self-check

Why has this example not required virtual yet?

Answer: It uses inherited and added methods without requiring dynamic dispatch through a base reference. Inheritance and runtime polymorphism are distinct.

9.2 Extension: inheritance, hiding, and virtual overriding

The original stops at protected and this. This labeled extension introduces virtual dispatch when a caller uses a base interface but needs derived-specific behavior. Merely sharing a method name does not create runtime polymorphism.

Declare the base method virtual and mark derived overrides override so the compiler checks matching signatures, including const. Calls through a base reference or pointer dispatch using the dynamic object type. Copying a derived object into a base value slices away its derived part.

Deleting a derived object through a base pointer normally requires a virtual base destructor. This differs from the original by-value Band examples. Choose a polymorphic design from the actual dispatch and lifetime requirements.

Complete classroom experiment

`L2_C09_D02.cpp` · CPP-Classroom source

```
#include <iostream>
#include <string>

struct Robot {
    virtual ~Robot() = default;
    virtual std::string kind() const { return "ROBOT"; }
};
struct Sentry : Robot {
    std::string kind() const override { return "SENTRY"; }
};

int main() {
    Sentry object;
    const Robot &ref = object;
    Robot sliced = object;
    std::cout << ref.kind() << ' ' << sliced.kind() << '\n';
    return 0;
}
```

Execution trace

The reference preserves the dynamic type and prints SENTRY; the sliced base value prints ROBOT.

Common error and its cause

Do not use `vector<Base>` for heterogeneous derived values whose dynamic behavior must be preserved.

Self-check

Can override detect a missing const on the derived method?

Answer: Yes. The mismatched override produces a compilation error.

9.3 Extension: value copying, moving, and resource ownership

Band contains strings, so default copying copies member values without custom destruction or copy operations. Prefer standard-library members that manage their own resources, a practice called the Rule of Zero. Learning classes does not require adding new and delete to every type.

unique_ptr expresses unique ownership. It is noncopyable but movable. A moved-from unique_ptr is guaranteed empty, while the destination takes ownership. Do not generalize that guarantee to every moved-from type; other library objects remain valid without necessarily having a specified value.

RAII ties cleanup to object lifetime. Fully constructed automatic objects are destroyed on normal scope exit and during exception unwinding. Deep-copy and clone challenges extend this idea only after the original Band value semantics are understood.

Complete classroom experiment

`L2_C09_D03.cpp` · CPP-Classroom source

```
#include <iostream>
#include <memory>
#include <utility>

int main() {
    std::unique_ptr<int> first = std::make_unique<int>(7);
    std::unique_ptr<int> second = std::move(first);
    std::cout << std::boolalpha << (first == nullptr) << ' ' << *second << '\n';
    return 0;
}
```

Execution trace

The result is true 7. Only second owns the integer, which is released automatically when its owner leaves scope.

Common error and its cause

Do not dereference first after ownership has moved from it; it is empty.

Self-check

Must Band implement a deep-copy function merely for completeness when all its members are strings?

Answer: No. Default member copying already provides the needed value semantics. Revisit special members when the class manages raw resources itself.

Chapter exercises

ID	Exercise
O21	Compose two motors
O22	Compose a battery and movement
O23	Inject a configuration
O24	A value snapshot
O25	Collect diagnostic messages
O31	Virtual dispatch
O32	A pure virtual interface
O33	Object slicing
O34	A virtual destructor
O35	A polymorphic collection
O36	Transfer unique ownership
O37	Rule of Zero
O38	Cleanup during exceptions
O39	A noncopyable device
O40	Declare and define members separately